

Distributed Data Processing with Apache Spark

A self-contained ~3-hour lecture + hands-on session: Architecture • RDDs vs. DataFrames • Lazy Evaluation • Shuffles • Large-Scale ETL

Agenda — ~3 Hours

Time	Block	Format
0:00 – 0:15	Hook: why a single machine can't process modern datasets	Lecture
0:15 – 0:35	Spark architecture: driver, cluster manager, executors, tasks	Lecture
0:35 – 1:00	RDDs vs. DataFrames vs. Spark SQL; lazy evaluation	Lecture + live demo
1:00 – 1:10	Break	—
1:10 – 1:35	Partitioning & shuffles; broadcast joins	Lecture + live demo
1:35 – 1:55	Case study: scaling a recommendation engine's feature pipeline	Discussion
1:55 – 2:50	Hands-on Exercise: large-scale ETL + broadcast join comparison	Lab
2:50 – 3:00	Wrap-up: cheat sheet & Q&A	Plenary

PREREQUISITES

Before You Start This Session

- Python 3.9+ with pyspark installed: `pip install pyspark==3.5.*`
- A Java 11 or 17 JRE on PATH — verify with `java -version` (PySpark requires this even though you write Python).
- A terminal, a code editor, and a web browser (for the Spark UI at <http://localhost:4040> once a job is running).
- The provided synthetic clickstream dataset (partitioned Parquet, generated ahead of time — do not generate it live).
- This deck is self-contained: every concept is followed by a runnable code snippet, and hands-on exercises (amber slides) reference the companion Jupyter notebook.

00

Why Distributed Data Processing?

What this section covers

- The scaling wall every AI team hits
- Vertical vs. horizontal scaling
- What Spark actually does about it

0.1 THE PROBLEM

A Single Machine Eventually Runs Out

500 GB+

typical size of a modern clickstream/event log dataset

16–64 GB

typical RAM on a laptop or single training VM

1

machine's worth of compute, no matter how large, has a ceiling

Discussion prompt: “You have a 500GB clickstream log and a laptop with 16GB RAM. `pandas.read_csv()` can't even load this file. What are your options?”

Vertical vs. Horizontal Scaling

Vertical Scaling

- Buy a bigger machine — more RAM, more cores, faster disk.
- Simple, but has a hard ceiling (and a steep cost curve near that ceiling).
- Eventually no single machine is big enough for the dataset.

Horizontal Scaling (Spark's Approach)

- Split the data across many ordinary machines.
- Run the SAME computation on every partition, in parallel.
- Scales by adding more machines — no theoretical ceiling.

01

Spark Architecture

What this section covers

- Driver, Cluster Manager, Executors, Tasks
- Why this split matters for debugging performance

How a Spark Job Is Actually Executed

1

Driver

Runs your main program;
builds the execution DAG;
coordinates everything else.

2

Cluster Manager

Allocates resources across the
cluster (standalone, YARN,
Kubernetes).

3

Executors

Worker processes on cluster
nodes; run tasks and hold data
partitions in memory/disk.

4

Tasks

The smallest unit of work —
one task processes exactly one
partition.

Diagnosing Slow Jobs Starts Here

- A job slow because the Driver is overloaded (e.g. too much data pulled back with `.collect()`) has a completely different fix than one slow due to executor-to-executor network shuffles.
- Symptom: Driver out-of-memory → check for `.collect()`, `.toPandas()`, or broadcast variables that are too large.
- Symptom: Jobs stuck in a few long-running tasks → check for data skew (one partition much larger than others).
- Symptom: Enormous time spent in “Exchange” stages in the query plan → check for unnecessary shuffles (Section 3).

02

RDDs, DataFrames & Lazy Evaluation

What this section covers

- Three abstractions, one execution engine
- Transformations vs. Actions
- Why laziness lets Spark optimize your code for you

2.1 THREE ABSTRACTIONS

RDDs vs. DataFrames vs. Spark SQL

Abstraction	What It Is	When to Use
RDD	Low-level distributed collection, no schema	Rarely needed directly today; unstructured data or full partitioning control
DataFrame	Distributed table with a schema, like a distributed pandas frame	Default choice for almost all ETL — optimized by Catalyst
Spark SQL	Standard SQL against DataFrames registered as views	Same execution engine as DataFrames — use whichever syntax reads better

Transformations Build a Plan; Actions Run It

```
from pyspark.sql import SparkSession, functions as F
spark = SparkSession.builder.appName("demo").getOrCreate()

df = spark.read.parquet("data/clickstream/")

# --- TRANSFORMATIONS: lazy, build a plan, touch NO data ---
clicks = df.filter(F.col("event_type") == "click")
grouped = clicks.groupBy("user_id").agg(F.count("*").alias("n"))

grouped.explain(True) # <-- the FULL plan already exists, nothing has run yet

# --- ACTION: this is the moment real computation happens ---
grouped.count()
```

Live demo: run `.explain(True)` BEFORE any action and show students the plan is already built. Then call `.count()` and point out this is the first moment data is actually touched.

Know Which Category Each Call Falls Into

Transformations (lazy)	Actions (trigger execution)
.filter() / .select() / .withColumn()	.count()
.groupBy() / .agg()	.collect()
.join()	.show()
.orderBy() / .repartition()	.write.parquet(...) / .write.csv(...)

Optimization You Get for Free

- Because the whole DAG is visible before execution, Spark's Catalyst optimizer can reorder, fuse, and prune operations automatically.
- Example: a `filter()` placed after a join in your code may be pushed BEFORE the join by the optimizer, if it's logically equivalent — filtering less data before the expensive join.
- This is impossible in eager, row-by-row execution (like plain pandas) — by the time pandas sees your filter, the join already happened.
- Practical implication: write readable code first; let the optimizer handle reordering, rather than manually micro-optimizing operation order.

03

Partitioning & Shuffles

What this section covers

- What a shuffle actually costs
- Choosing a sensible partition count
- Broadcast joins: avoiding the shuffle entirely

The Single Most Expensive Operation in Spark

- A DataFrame is split into partitions; each partition is processed independently by one task on one executor core.
- Operations like groupBy, join, and repartition often require a shuffle — redistributing data across the network so related rows land on the same executor.
- A shuffle involves disk I/O, serialization, AND network transfer across the whole cluster — it is almost always the dominant cost in a Spark job.
- Rule of thumb: any time rows need to be compared or combined ACROSS partitions (not within one), expect a shuffle.

Too Few vs. Too Many Partitions

```
# Spark's out-of-the-box default is 200 shuffle partitions —  
# usually far too many for a laptop-scale job, far too few for a large cluster  
spark.conf.set("spark.sql.shuffle.partitions", "8") # tune for YOUR cluster size  
  
# Too few partitions -> the cluster sits idle, only a few cores busy  
# Too many partitions -> task scheduling overhead can exceed the actual work per task  
  
df.rdd.getNumPartitions() # inspect current partition count  
df.repartition(16)        # explicitly repartition (causes a shuffle!)  
df.coalesce(4)            # reduce partitions WITHOUT a full shuffle
```

coalesce() is cheaper than repartition() when reducing partition count, because it avoids a full shuffle by merging existing partitions locally.

3.3 BROADCAST JOINS

Avoiding the Shuffle Entirely

```
from pyspark.sql.functions import broadcast

# Naive join -- Spark may shuffle BOTH tables (expensive if `features` is huge)
joined_naive = features.join(course_catalog_df, "course_id")

# Broadcast join -- sends the SMALL table to every executor's memory,
# so the huge `features` table never needs to be shuffled at all
joined_fast = features.join(broadcast(course_catalog_df), "course_id")

# Compare the physical plans:
joined_naive.explain() # look for "Exchange" (shuffle) stages
joined_fast.explain()  # look for "BroadcastExchange" instead
```

Use broadcast() when one side of a join is small enough to fit comfortably in executor memory (a common rule of thumb: well under ~200MB).

04

Case Study

What this section covers

- Scaling a recommendation engine's feature pipeline
- From pandas to Spark, step by step

4.1 CASE STUDY

From 50,000 Users to 5 Million: Scaling a Feature Pipeline

- Scenario: a course-recommendation startup's nightly feature pipeline (clicks per course, avg session length) runs in 20 minutes on pandas with 50,000 users.
- At 5 million users, the same pandas script now takes 14 hours and frequently runs out of memory.
- Root cause: pandas loads the ENTIRE dataset into a single process's memory — there is no horizontal scaling path.
- Fix: migrate the groupBy/aggregate logic to Spark DataFrames — identical logic, now distributed across many executors.
- Remaining cost: the groupBy("course_id") aggregation still requires a shuffle — mitigated by partitioning upstream event logs by course_id, and writing output as partitioned Parquet for efficient downstream reads.

Large-Scale ETL + Broadcast Join Comparison

1. Open the companion notebook: `Lecture1_Spark_ETL_Exercise.ipynb`.
2. Start a local `SparkSession` and read the provided partitioned clickstream Parquet dataset.
3. Build a filter → `groupBy` → `agg` pipeline computing `click_count` and `avg_session_length` per (`user_id`, `course_id`).
4. Call `.explain(True)` before any action and identify the shuffle-inducing stage in the plan.
5. Write the result as partitioned Parquet, then join it against the small `course_catalog` table twice: once naively, once with `broadcast()`.
6. Compare shuffle read/write bytes for both joins using the Spark UI (<http://localhost:4040>).

Deliverable: A completed notebook showing the ETL pipeline, an annotated `.explain()` output identifying the shuffle stage, and a before/after shuffle-bytes comparison for the two joins.

Apache Spark Cheat Sheet

Task	Code
Start a session	<code>SparkSession.builder.appName(...).getOrCreate()</code>
Read Parquet	<code>spark.read.parquet(path)</code>
Inspect the plan (no execution)	<code>df.explain(True)</code>
Trigger execution	<code>df.count() / df.collect() / df.write...</code>
Filter / aggregate	<code>df.filter(...).groupBy(...).agg(...)</code>
Tune shuffle partitions	<code>spark.conf.set("spark.sql.shuffle.partitions", n)</code>
Cheap partition reduction	<code>df.coalesce(n)</code>
Avoid shuffling a big table	<code>big.join(broadcast(small), key)</code>
Write partitioned output	<code>df.write.partitionBy(col).parquet(path)</code>

KEY TAKEAWAY

Distribute the computation, not just the data:

Partitions + Lazy Evaluation + Minimal Shuffles = a Spark job that actually scales

Every lever in this lecture — partition count, broadcast joins, output format — exists to reduce the one truly expensive operation: the shuffle.

Next: Lecture 2 — Distributed Compute & Training with Ray.